Interactive Statistical Analysis of Personal Running Data Using Strava Exports

Gian Federspiel
Informatik
HSLU Lucerne University of Applied Sciences and Arts
Rotkreuz, Switzerland
gian.federspiel@stud.hslu.ch

Seedorf Asamoah
Informatik
HSLU Lucerne University of Applied Sciences and Arts
Rotkreuz, Switzerland
seedorf.asamoah@stud.hslu.ch

***Abstract*—This project presents an interactive web dashboard for the statistical analysis of personal running data exported from the Strava fitness platform. Users upload their own activity CSV file, which the application cleans and filters to running activities. The dashboard computes descriptive statistics and applies inferential methods including Pearson correlation and ordinary least squares linear regression to quantify the relationship between heart rate and running pace, and to assess whether the athlete's heart-rate-adjusted pace has improved over time. All results are visualised interactively using Plotly charts embedded in a Streamlit application.**

***Index Terms*—running statistics, Strava, linear regression, Pearson correlation, Streamlit, data visualisation**

I. Introduction

Recreational running has become increasingly popular, and modern fitness trackers such as Strava record rich per-activity data including distance, moving time, and heart rate. While Strava itself provides aggregate summaries, it does not expose the underlying statistical relationships in a user-controllable way. This project fills that gap by providing an open-source dashboard that allows any Strava user to upload their own data export and immediately obtain statistically grounded insights about their training.

The primary research questions addressed by the dashboard are:

- Has the athlete's running pace improved over time *at the same heart rate*, and is the trend statistically significant?
- Is there a statistically significant linear relationship between average heart rate and running pace?

II. Data

A. Source and Format

Data is obtained via Strava's built-in export feature (Settings → My Account → Download or Delete Your Account → Request Your Archive). The downloaded archive contains an `activities.csv` file with one row per recorded activity and columns including activity date, type, distance, moving time, average heart rate, and elevation gain.

B. Preprocessing

The data processing pipeline (implemented in `data_processor.py`) performs the following steps:

1) **Column normalisation:** Column names are lower-cased and mapped to a consistent internal schema to handle differences between Strava export versions.
2) **Activity filtering:** Only activities of type *Run* or *VirtualRun* are retained.
3) **Unit conversion:** Distance values stored in metres (median > 200) are converted to kilometres. Moving time strings in `H:MM:SS` or `M:SS` format are parsed to seconds.
4) **Pace computation:** Running pace (min/km) is derived as pace $= \frac{\text{moving time [min]}}{\text{distance [km]}}$. Implausible values outside $[3, 15]$ min/km are set to missing.
5) **Temporal features:** ISO week start and calendar month start columns are added for aggregation.

III. Statistical Methods

A. Descriptive Statistics

The dashboard reports the total number of runs, cumulative distance, mean pace, best (minimum) pace, longest single run, and mean heart rate across all runs. A rolling average over a window of 10 runs is overlaid on the pace-over-time chart, providing a smoothed view of the athlete's training trajectory without being overly sensitive to individual outlier sessions.

B. Weekly Volume

Weekly running volume is computed by summing distances within each ISO calendar week and displayed as a bar chart, giving an immediate impression of training consistency and load distribution.

C. Heart Rate Analysis

For runs that include heart rate data, the dashboard provides two complementary views. First, a histogram with 25 bins shows the distribution of average heart rate across all runs, along with summary statistics (mean, minimum, maximum, and standard deviation). Second, a scatter plot with an OLS trend line shows how average heart rate evolves over time, giving a first impression of whether typical training intensity is drifting up or down.

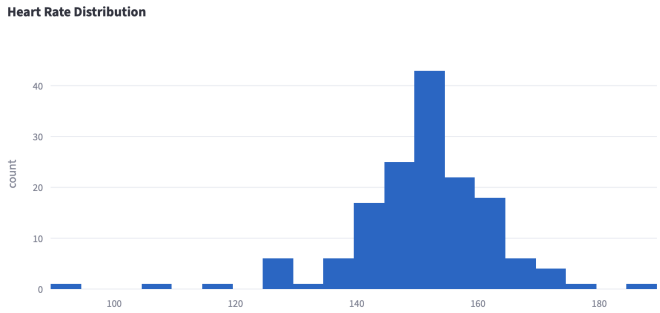

Fig. 1. Heart rate distribution across all recorded runs. The distribution is roughly unimodal and right-skewed, with the majority of runs falling between 140 and 170 bpm and a clear peak around 150–155 bpm.

D. Pearson Correlation: Pace vs Heart Rate

To quantify the relationship between cardiovascular load and running speed, the Pearson correlation coefficient r is computed between average heart rate (bpm) and pace (min/km) across all runs with heart rate data:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}} \tag{1}$$

A two-sided t-test is applied to assess significance at $\alpha = 0.05$. The scatter plot includes an OLS trendline, and the dashboard reports r, R^2, p, and a plain-language interpretation of the direction (e.g. *higher HR* $\rightarrow$ *slower pace*). Fig. 1 shows the heart rate distribution from the example dataset. Fig. 2 shows the corresponding pace regression.

E. Linear Regression: HR-Adjusted Pace Improvement over Time

A naive regression of pace on time conflates true fitness gains with the trivial effect of running easier or harder sessions. To isolate genuine improvement, a two-stage procedure is used. First, the OLS fit of pace on heart rate from the previous subsection is used to compute, for each run i, the *expected pace* $\hat{p}_i$ given that run's average heart rate. The pace residual is then

$$e_i = p_i - \hat{p}_i, \tag{2}$$

representing how much faster ($e_i < 0$) or slower ($e_i > 0$) the athlete ran than would be expected for the observed heart rate.

In the second stage, a simple linear regression is fitted with sequential run number as the predictor and the pace residual as the response:

$$\hat{e}_i = \beta_0 + \beta_1 \cdot i \tag{3}$$

where i is the run index (1-based). A negative slope β_1 indicates that the athlete is, over time, running faster than expected for the same heart rate — the operational definition of improving aerobic fitness. Combined with the observed run frequency, a monthly residual change is derived. Statistical significance is assessed at $\alpha = 0.05$ via the F-test p-value. All regression computations use `scipy.stats.linregress` [1].

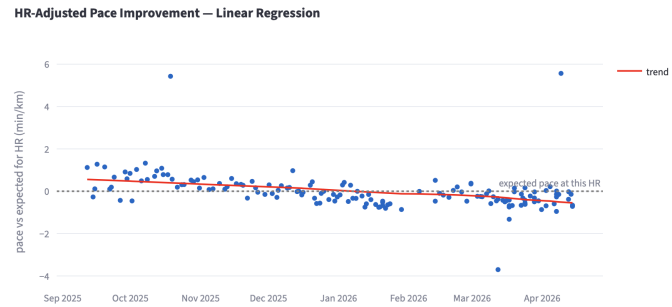

Fig. 2. HR-adjusted pace improvement over time (Sep 2025 – Apr 2026). Each point is the pace residual for one run after removing the effect of heart rate; the red line is the OLS regression fit. Points below the dotted zero line correspond to runs that were faster than expected for the observed heart rate.

IV. Implementation

The application is built in Python 3 using Pandas [2] for data manipulation, SciPy [1] for statistical computations, Plotly Express [4] for interactive charts, and Streamlit [3] for the web interface. The codebase is split into `data_processor.py` (ingestion and feature engineering) and `app.py` (layout and visualisation). Data is processed in memory and stored in Streamlit session state — nothing is written to disk or sent to external servers.

V. Discussion and Conclusion

The dashboard demonstrates that meaningful statistical analysis of personal fitness data is achievable with a concise, well-structured codebase. The combination of rolling averages, correlation analysis, and linear regression gives athletes an evidence-based view of their progress beyond simple cumulative totals.

A key limitation remains that the models do not control for confounders such as terrain, elevation gain, or weather. The pace-improvement model accounts for heart rate but treats all other run-to-run variation as noise. Future work could incorporate multiple regression to address this, and direct OAuth integration with the Strava API would remove the manual export step.

Despite these limitations, the project achieves its goal of making statistical running analysis accessible to any Strava user through an intuitive, privacy-respecting, browser-based interface.

References

[1] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
[2] W. McKinney, "Data structures for statistical computing in Python," *Proc. 9th Python in Science Conf.*, pp. 56–61, 2010.
[3] Streamlit Inc., "Streamlit — A faster way to build and share data apps," 2024. [Online]. Available: https://streamlit.io
[4] Plotly Technologies Inc., "Collaborative data science," 2024. [Online]. Available: https://plotly.com